Edge cases (an ambiguous routing decision) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `supervisor_routing_practice.py`: routing looks fine until two specialists both plausibly fit a task. This file makes that happen on purpose, counts how often the supervisor wobbles, and adds a way to recover. **If not:** Project 4's Research-vs-Analysis boundary would misroute in production, with no test that ever showed it.

Solution

Story: one run per task can't tell a stable choice from a coin flip; three runs of the same task can. **If not:** an inconsistent route would pass every one-shot test and fail at random for users.

```
# supervisor_routing_practice.py
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END


class RouteState(TypedDict):
    task: str
    picked: str


def supervisor(state: RouteState) -> dict:
    # temperature > 0 on purpose, so an ambiguous task can wobble between
    # runs — which is exactly what this exercise measures.
    model = ChatOpenAI(model="gpt-4o-mini", temperature=1.0)
    prompt = (
        "Route this task to exactly one specialist. Reply with only "
        "'research_agent' (finding out facts or causes) or "
        "'analysis_agent' (interpreting data we already have).\n\n"
        f"Task: {state['task']}"
    )
    choice = model.invoke(prompt).content.strip()
    if "analysis" in choice:
```

```
        return {"picked": "analysis_agent"}
    return {"picked": "research_agent"}


def build_graph():
    builder = StateGraph(RouteState)
    builder.add_node("supervisor", supervisor)
    builder.add_edge(START, "supervisor")
    builder.add_edge("supervisor", END)
    return builder.compile()


graph = build_graph()


def run_ambiguity_check(prompts: list[str], attempts: int = 3) -> dict:
    tally = {}
    for prompt in prompts:
        picks = []
        for _ in range(attempts):
            result = graph.invoke({"task": prompt, "picked": ""})
            picks.append(result["picked"])
        tally[prompt] = picks
    return tally

if __name__ == "__main__":
    test_prompts = [
        "find out why sales dropped last quarter",
        "what does this survey data actually tell us",
        "look into the causes of the outage",
        "explain what happened to our conversion rate",
        "dig into the reasons behind the delay",
    ]
    tally = run_ambiguity_check(test_prompts)
    for prompt, picks in tally.items():
        consistent = len(set(picks)) == 1
        label = "consistent"
        if not consistent:
            label = "INCONSISTENT"
        print(f"{prompt!r}:")
        print(f"  {picks} ({label})")
```

Expected output (exact picks vary by run):

```
'find out why sales dropped last quarter':
  ['research_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
```

```
'what does this survey data actually tell us':
  ['analysis_agent', 'analysis_agent', 'analysis_agent'] (consistent)
'look into the causes of the outage':
  ['research_agent', 'research_agent', 'research_agent'] (consistent)
'explain what happened to our conversion rate':
  ['analysis_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
'dig into the reasons behind the delay':
  ['research_agent', 'research_agent', 'research_agent'] (consistent)
```